7.1.2 Analyzers to the rescue

To build an intelligent search engine that never disappoints, we give the engine extra assistance during data indexing by using *text analysis* carried out by software modules called *analyzers*. To serve many queries, we must prepare the search engine on the data it consumes during the indexing phase.

> **Only text fields are analyzed—the rest are not!**
>
> Elasticsearch analyzes only `text` fields before storing them in their respective inverted indexes. No other data types undergo text analysis. Elasticsearch also uses the same principle of analyzing the query's text fields when executing the search query.

Users don't often want to enter an exact search string—they may omit some common words, reverse the order, ignore correct spellings etc. The power of a search engine like Elasticsearch comes from the fact that it can search not only for individual words but also for synonyms, abbreviations, root words, and so on. We can attach different analyzers—including standard, language, and custom analyzers—based on our requirements.

Analyzing data during the indexing process and capturing it based on our requirements lets Elasticsearch satisfy a spectrum of search query variations. An analyzer consists of several other components that help analyze text.

7.2 Analyzer modules

An analyzer is a software module tasked with two functions: *tokenization* and *normalization*. Elasticsearch uses these processes so `text` fields are analyzed and stored in inverted indexes for query matching. Let's look at these concepts at a high level before drilling down into the anatomy of the analyzer.

7.2.1 Tokenization

Tokenization, as the name indicates, is a process of splitting sentences into individual words by following certain rules. For example, we can instruct the process to break sentences at a delimiter, such as whitespace, a letter, a pattern, or other criteria.

This process is carried out by a component called a *tokenizer*, whose sole job is to chop the sentence into individual words called *tokens* by following certain rules. A `whitespace` tokenizer is commonly employed during the tokenization process: it separates each word in the sentence with whitespace and removes any punctuation and other non-characters.

Words can also be split based on non-letters, colons, or other custom separators. For example, a movie reviewer's assessment saying "The movie was sick!!! Hilarious :) :)" can be split into individual words: *The*, *movie*, *was*, *sick*, *Hilarious*, and so on (note that the words are not yet lowercased). Similarly, *pickled-peppers* can be tokenized to *pickled* and *peppers*, *K8s* can be tokenized to *K* and *s*, and so on.